Embedded
Online
Conference

# Exception Handling

**Nathan** Jones

www.embeddedonlineconference.com

# “Exceptions”, not “Exceptions”

## Exceptions

- Possible (though, perhaps, unlikely) edge case that would disrupt normal program execution if left unaddressed
- Ex:
  - Numeric overflow/ underflow
  - Invalid user input/ sensor values

- **try/catch**
- Return codes
- Out-arguments
- Local variable (i.e. `errno`)
- Signal handlers/Callbacks

C++ (and Java, Python, etc) exception handling mechanism

# Exceptions, not Errors

## Exceptions

- Possible (though, perhaps, unlikely) edge case that would disrupt normal program execution if left unaddressed
- Ex:
  - Numeric overflow/ underflow
  - Invalid user input/ sensor values

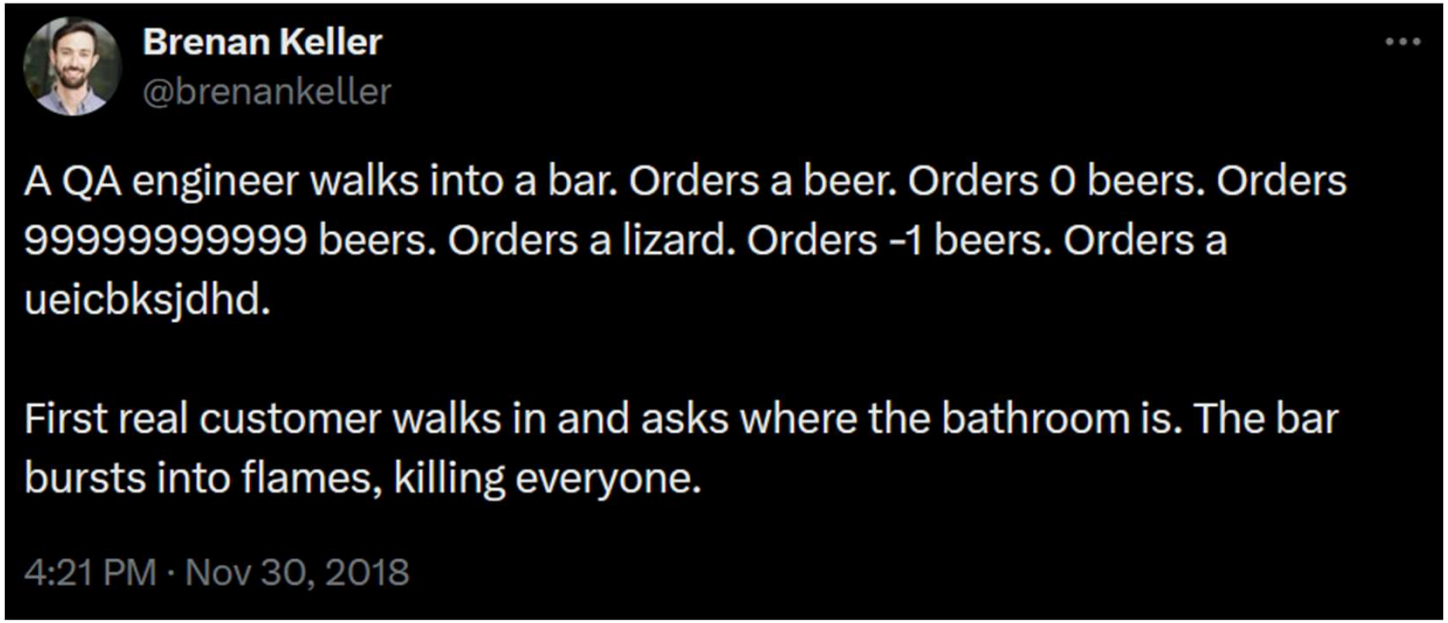
## Errors

- Incorrect code; “impossible” situation
- Ex:
  - `int *p; *p = 4;`
  - `int i = -1; sqrt(i);`
  - `xTaskCreate` returns error

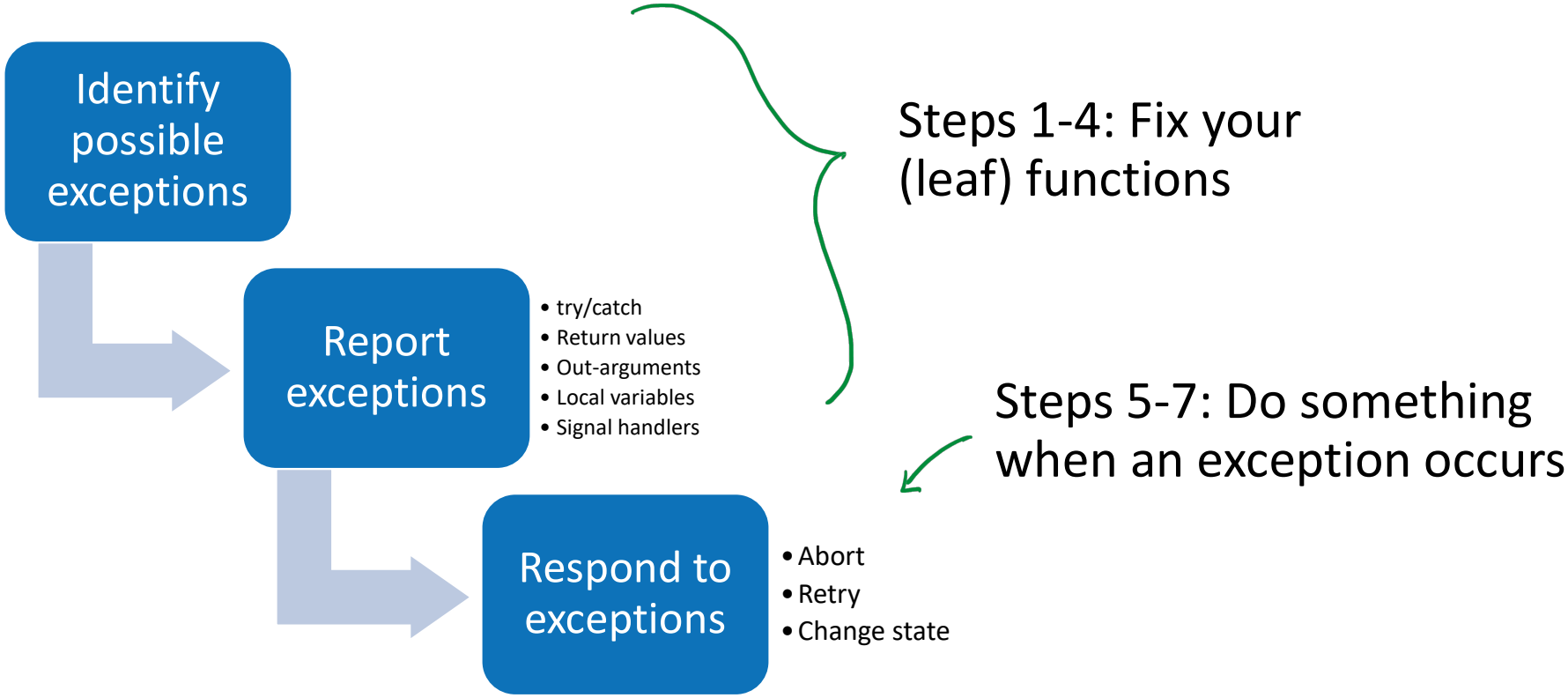
Use asserts, i.e.

- Reset the system
- Revert to a “safe mode”

# Robust programs are “anti-fragile”



# Agenda



# A Case Study

```
void myTask(void)
{
  while(1)
  {
    uint16_t val = A();
    B(val);
    C();
  }
}
```

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  return /* ADC result */;
}
```

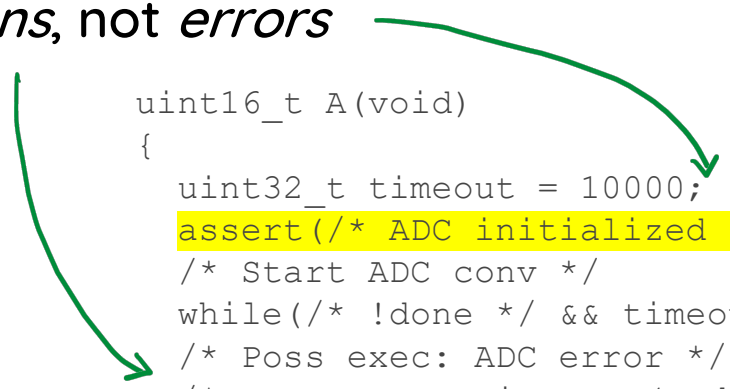
# 1a) Identify which exceptions to handle

```
void myTask(void)
{
  while(1)
  {
    uint16_t val = A();
    B(val);
    C();
  }
}
```

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done) */
  return /* ADC result */;
}
```

# 1a) Identify which exceptions to handle

Remember: *exceptions*, not *errors*



```
void myTask(void)
{
  while(1)
  {
    uint16_t val = A();
    B(val);
    C();
  }
}
```

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  assert(/* ADC initialized */);
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done) */
  return /* ADC result */;
}
```

# Your turn!

Come up with as many examples of exceptions as you can!

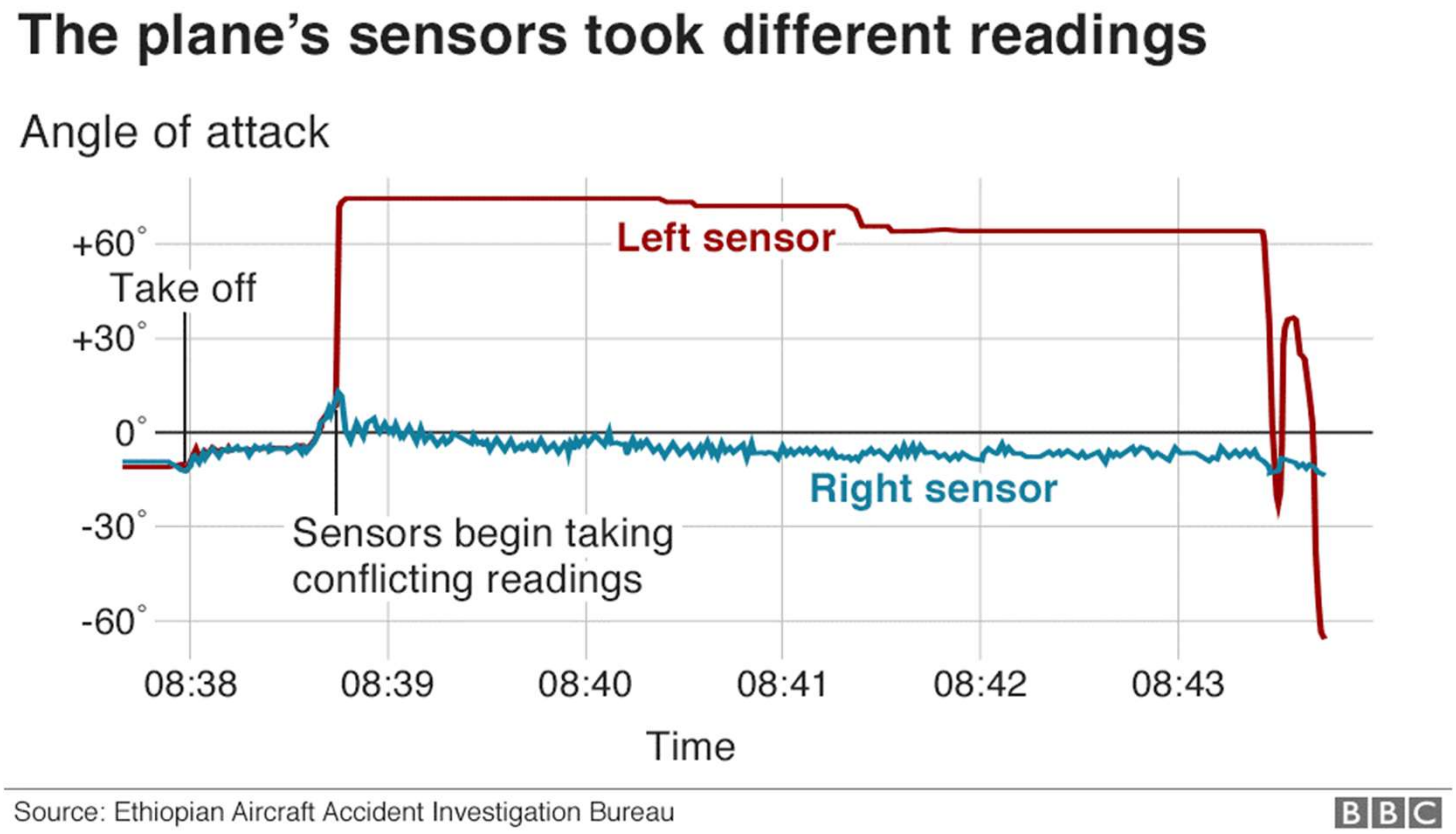
- One of your previous/current projects
- From these sample systems
  - https://wokwi.com/projects/430777548539551745
  - https://wokwi.com/projects/430842301912312833
  - https://www.tinkercad.com/things/7TYqSek80uE-eoc2025exception-handling-workshoptinkercad?sharecode=89255xc2210w49tafGJeKD75uj5ZI1LUb-5E1cqrbjE

# Examples of Possible Exceptions

- Numeric overflow/underflow
- Invalid user input/sensor values
- Communications errors (parity error, checksum failed, timeout, etc)
- Odd behavior (ADC maxing out, sensor reading is physically impossible*, full power to motor for a long period of time, etc)
- Peripheral not connected/initialized
- File not found, empty file
- Nice to haves:
    - Debugger attached/not attached
    - USB-to-UART attached/not attached

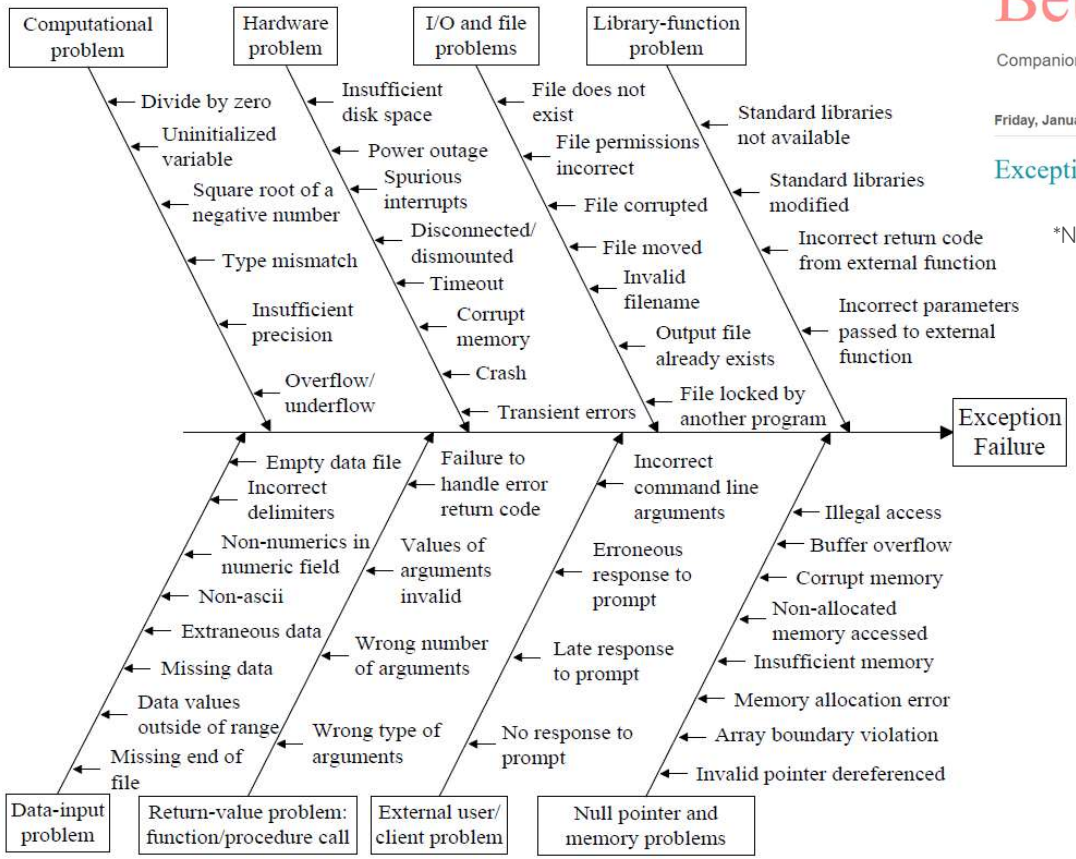
*: One possible source of the Boeing 737-MAX crash!

# Examples of Possible Exceptions



# CHILDREN are your problem

https://betterembsw.blogspot.com/2013/01/exception-handling-fishbone-diagram.html




Better Embedded System SW

Companion blog to the book Better Embedded System Software by Phil Koopman at Carnegie Mellon Univers

Friday, January 25, 2013

Exception Handling Fishbone Diagram

Pages

Home

Automotive softwar

*NOTE: I would classify some of these as *errors*, not exceptions

# 1b) *Prepare* to report the exception

```
void myTask(void)
{
  while(1)
  {
    uint16_t val = A();
    B(val);
    C();
  }
}
```

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done) */
  return /* ADC result */;
}
```

# Reporting Strategies

| | |
|---|---|
| **Try/catch** | `int foo( ARGS ) throw() {`<br>`  if( /* error */) throw /* Error type */;`<br>`}` |
| **Return value** | `[[nodiscard]] err_t foo( ARGS ) {`<br>`  err_t err = /* "No error" */;`<br>`  if(/* error */) err = /* Error type */;`<br>`  return err;`<br>`}` |
| **Out-argument** | `int foo( ARGS, err_t * err ){`<br>`  if(/* error */) *err = /* Error type */;`<br>`}` |
| **Local variable** | `int foo( ARGS ) {`<br>`  if(/* error */) errno = /* Error type */;`<br>`}` |
| **Signal/Callback** | `int foo( ARGS, void*(*err_cbk)(void*) ) {`<br>`  if(/* error */) err_cbk(NULL);`<br>`}` |

- Error reporting is part of the API!
- Report errors at the appropriate level of abstraction
- err_t/errno
  - Boolean
  - enum/int
  - Struct
  - "sum type", e.g.
    - std::optional
    - std::expected
    - tagged union

# Reporting Strategies

| Use… | …if… |
|---|---|
| **Try/catch** | • You want to *ensure* that your code handles all exceptions (and you don't mind your program being terminated if an exception isn't caught!)<br>• You need to return an error from inside a constructor<br>• ***Provided*** you can afford non-deterministic time/memory |
| **Return value** | • You want simple, deterministic exception handling<br>• You have control over your function signatures |
| **Out-argument** | • You need to preserve the return value of your functions |
| **Local variable** | • You're working on legacy code and can't change your function signatures |
| **Signal/Callback** | • There is a common fix/workaround to specific exceptions (think peripheral error ISR)<br>• You want a single point of truth for exception handling |

- Error reporting is part of the API!
- Report errors at the appropriate level of abstraction
- err_t/errno
  - Boolean
  - enum/int
  - Struct
  - "sum type", e.g.
    - std::optional
    - std::expected
    - tagged union

# Advantages and Disadvantages

| | Advantages | Disadvantages |
|---|---|---|
| **Try/catch** | • Separates "happy path" from exception handling code (?)<br>• Calling code can't ignore the error<br>• Only way to return an error from inside a constructor | • Uncaught exceptions terminate the program<br>• The destructor for an incompletely constructed object isn't called, possibly creating a memory leak.<br>• Stateless (unless you create your own exception types) |
| **Return value** | • Simple<br>• Program is alerted to error condition immediately after one occurs (stateful) | • Moves any actual return values to out-arguments<br>• Can be ignored (but [[nodiscard]] could help) |
| **Out-argument** | • Preserves function's return value | • Function signatures all need to change<br>• Needs to be reset before each time its used<br>• Can be ignored |
| **Local variable** | • Does not co-opt return value, nor require changing function signatures | • Needs to be reset before each time its used<br>• Can be ignored<br>• Thread-safety needs to be considered carefully<br>• Global variables are generally discouraged |
| **Signal/ Callback** | • Singular function for error handling | • Possibly some state information is lost<br>• Doesn't (immediately) alter program flow |

# 1b) *Prepare* to report the exception

```
void myTask(void)
{
  while(1)
  {
    uint16_t val = A();
    B(val);
    C();
  }
}
```

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done)
  return /* ADC result */;
}
```

# 1b) *Prepare* to report the exception

```
void myTask(void)
{
  while(1)
  {
    uint16_t val = A();
    B(val);
    C();
  }
}
```

Storing return values of functions

```
void myTask(void)
{
  while(1)
  {
    uint16_t val;
    err_t err = A(&val);
    err = B(&val);
    err = C();
  }
}
```

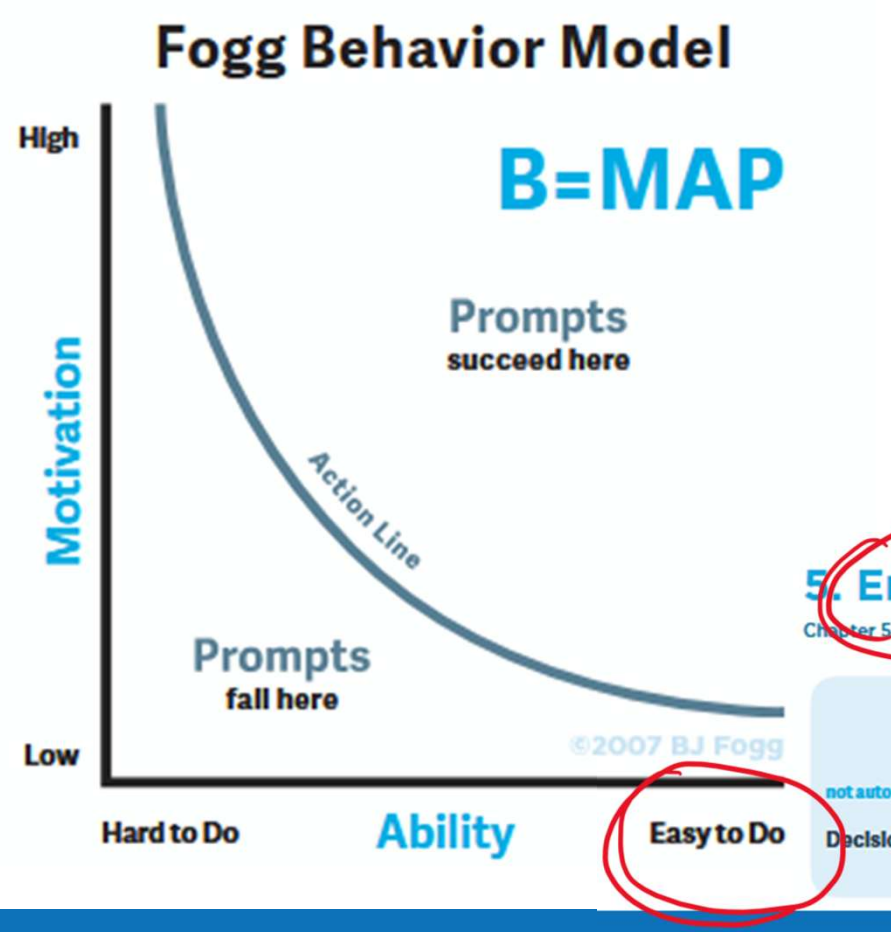
```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done)
  return /* ADC result */;
}
```

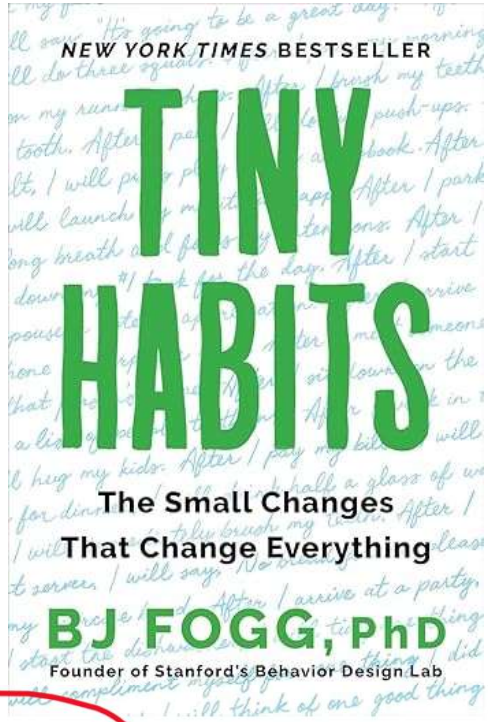
Using return value for error code

```
[[nodiscard]] err_t A(uint16_t * val)
{
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done)
  *val = /* ADC result */;
  return err;
}
```

# 2) Celebrate!

# 2) Celebrate!




# 2) Celebrate!

```
void myTask(void)
{
  while(1)
  {
    uint16_t val;
    err_t err = A(&val);
    err = B(&val);
    err = C();
  }
}
```

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*****************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *****************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done)
  *val = /* ADC result */;
  return err;
}
```

# Your turn!

Prepare to handle exceptions and then celebrate!

- One of your previous/current projects
- From these sample systems
  - https://wokwi.com/projects/430777548539551745
  - https://wokwi.com/projects/430842301912312833
  - https://www.tinkercad.com/things/7TYqSek80uE-eoc2025exception-handling-workshoptinkercad?sharecode=89255xc2210w49tafGJeKD75uj5ZI1LUb-5E1cqrbjE

# 3a) Log the exception

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);

  /* Poss exec: ADC error */
  /* Poss exec: Timeout */
  if(/* ADC error */ || timeout == 0)
  {
    ...
  }

  *val = /* ADC result */;
  return err;
}
```

# 3a) Log the exception

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);

  /* Poss exec: ADC error */
  /* Poss exec: Timeout */
  if(/* ADC error */ || timeout == 0)
  {
    ...
  }

  *val = /* ADC result */;
  return err;
}
```

## Increment a variable

```
uint32_t err = 0;
if(/* there was an error */) err++;
```

## Turn on/Blink an LED

```
if(/* error */) HAL_GPIO_SetPin(LED);
if(/* error */) xTaskResume(BlinkyTask);
```

## Store to memory

```
if(/* sensor value out-of-range error */){
  dataFile = SD.open("/data.txt", FILE_APPEND);
  if(datafile) {...}
}
```

## Send out over UART/USB

```
if(/* sensor value out-of-range error */){
  Serial.println("Sensor val out of range");
}
```

# 3a) Log the exception

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*******************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *******************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);

  /* Poss exec: ADC error */
  /* Poss exec: Timeout */
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
  }

  *val = /* ADC result */;
  return err;
}
```

## Increment a variable

```
uint32_t err = 0;
if(/* there was an error */) err++;
```

## Turn on/Blink an LED

```
if(/* error */) HAL_GPIO_SetPin(LED);
if(/* error */) xTaskResume(BlinkyTask);
```

## Store to memory

```
if(/* sensor value out-of-range error */){
  dataFile = SD.open("/data.txt", FILE_APPEND);
  if(datafile) {...}
}
```

## Send out over UART/USB

```
if(/* sensor value out-of-range error */){
  Serial.println("Sensor val out of range");
}
```

# 3b) Raise the alarm

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = MODA_NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = MODA_ERR_SOMETHING;
  }
  *val = /* ADC result */;
  return err;
}
```

```
/*** moduleA.h ***/
typedef enum {
  MODA_NO_ERR,
  MODA_ERR_SOMETHING,
  MODA_ERR_ANOTHER_THING,
  NUM_ERRS_MODA
} err_t;

err_t A(uint16_t * val);
```

# 3b) Raise the alarm

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = MODA_NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = MODA_ERR_SOMETHING;
  }
  *val = /* ADC result */;
  return err;
}
```

```
/*** moduleA.h ***/
typedef enum {
  MODA_NO_ERR,
  MODA_ERR_SOMETHING,
  MODA_ERR_ANOTHER_THING,
  NUM_ERRS_MODA
} err_t;

err_t A(uint16_t * val);
```

Alternatively:

- throw ___ (try/catch)
- return ___ (return values, early return)
- *err = ___ (out-args)
- err = ___ (local var)

# 3c) End-of-function (1/2): What value to output?

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*******************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *******************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = ERR_SOMETHING;
  }
  else *val = /* ADC result */;
  return err;
}
```

1. -1/0/NULL
    1. But [“Avoid in-band errors” (SEI ERR02-C)]
2. Last valid value

```
uint16_t A(err_t * err){
  uint16_t last = 0;
  // ...
  if(/* ADC error */ || timeout == 0){...}
  else last = /* ADC result */;
  return last;
}
```

3. Closest legal

```
if(/* ADC result */ > 0xFFF) *val = 0xFFF;
```

# End-of-function (2/2): Clean-up (in C)

**Using gotos**

```
if      (/* malloc failure */)    {goto one;}
else if (/* couldn't open file */){goto two;}
else     err = C();

fclose();
two:
  free();
one:
  // default?
```

**Using switch/case (with intentional fall-through)**

```
if      (/* malloc failure */)    {err = 1;}
else if (/* couldn't open file */){err = 2;}
else     err = C();

fclose();
switch(err){
  case 2:
    free();
  case 1:
  default:
    // default?
}
```

# 4) Celebrate!

# 4) Celebrate!

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  *  /\_/\  Attack Cat says this function is ready *
  * ( o.o ) to pounce on possible exceptions      *
  *  > ^ <  and it won't pass along bad data!     *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = ERR_SOMETHING;
  }
  else *val = /* ADC result */;
  return err;
}
```

# Review

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  return /* ADC result */;
}
```

# Review

## Identified possible exceptions

```
uint16_t A(void)
{
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout */
  return /* ADC result */;
}
```

# Review

## Prepared to report exceptions

```
[[nodiscard]] err_t A(uint16_t * val)
{
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done)
  *val = /* ADC result */;
  return err;
}
```

# Review

## Celebrated!

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  /* Poss exec: ADC error */
  /* Poss exec: Timeout (and ADC !done)
  *val = /* ADC result */;
  return err;
}
```

# Review

## Logged the exception

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
  }

  *val = /* ADC result */;
  return err;
}
```

# Review

## Raised the alarm

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = ERR_SOMETHING;
  }
  *val = /* ADC result */;
  return err;
}
```

# Review

## Fixed the output

```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*********************************************
  * (\(\     Sus Bunny says this function has   *
  * ( -.-)   identified possible exceptions and *
  * o_(")(") she is giving them mad side-eye!   *
  *********************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = ERR_SOMETHING;
  }
  else *val = /* ADC result */;
  return err;
}
```

# Review

## Celebrated!

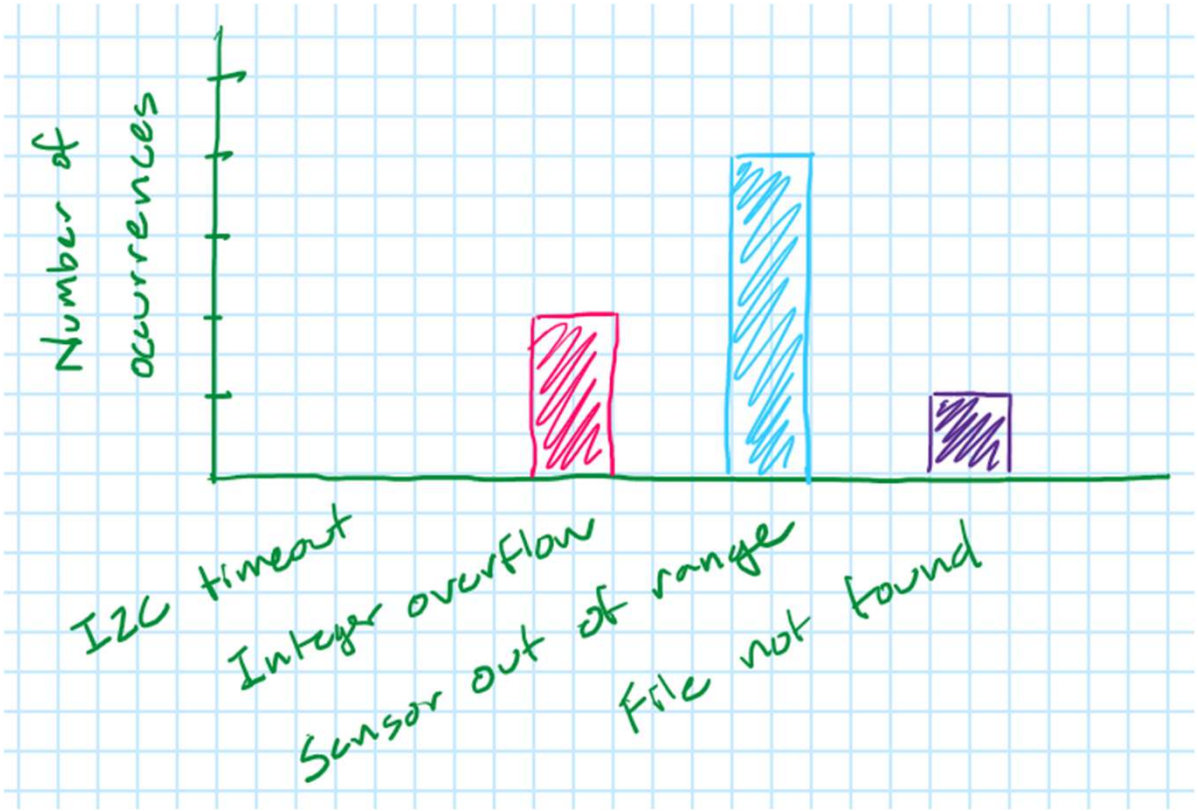
```
[[nodiscard]] err_t A(uint16_t * val)
{
  /*************************************************
  *  /\_/\  Attack Cat says this function is ready *
  * ( o.o ) to pounce on possible exceptions       *
  *  > ^ <  and it won't pass along bad data!      *
  *************************************************/
  err_t err = NO_ERR;
  uint32_t timeout = 10000;
  /* Start ADC conv */
  while(/* !done */ && timeout-- > 0);
  if(/* ADC error */ || timeout == 0)
  {
    HAL_GPIO_SetPin(LED);
    err = ERR_SOMETHING;
  }
  else *val = /* ADC result */;
  return err;
}
```

# Your turn!

Report your exceptions and then celebrate!

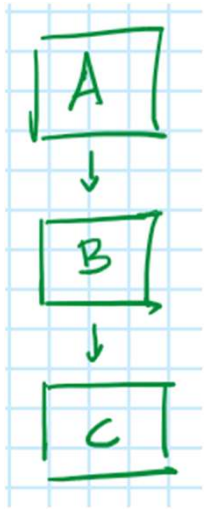
- One of your previous/current projects
- From these sample systems
  - https://wokwi.com/projects/430777548539551745
  - https://wokwi.com/projects/430842301912312833
  - https://www.tinkercad.com/things/7TYqSek80uE-eoc2025exception-handling-workshoptinkercad?sharecode=89255xc2210w49tafGJeKD75uj5ZI1LUb-5E1cqrbjE

# 5) (Optional) Take data

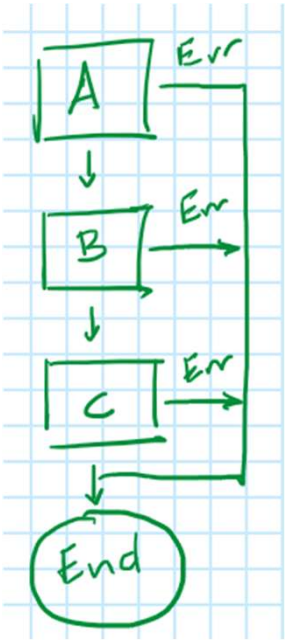


# 6a) Handle the exception

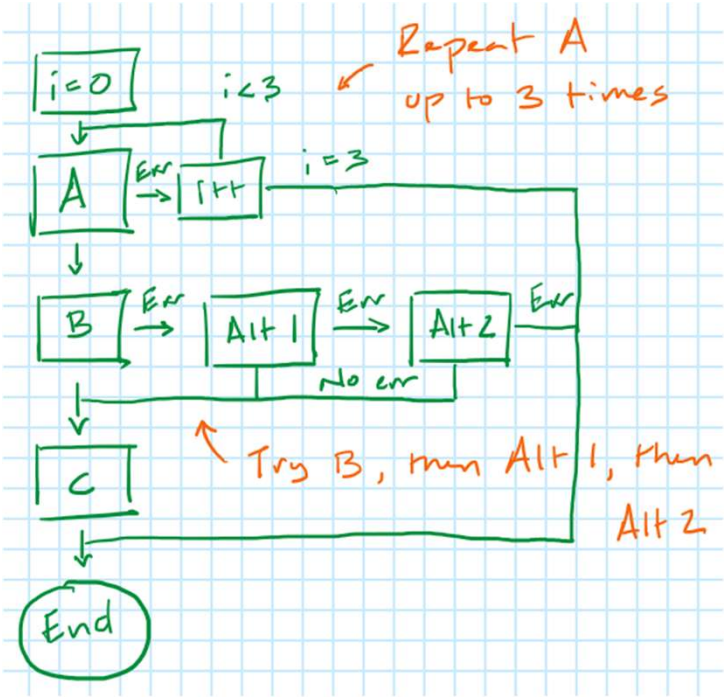
```
void myTask(void) {
  while(1) {
    uint16_t val
    err_t err = A(&val);
    err = B(&val);
    err = C();
  }
}
```



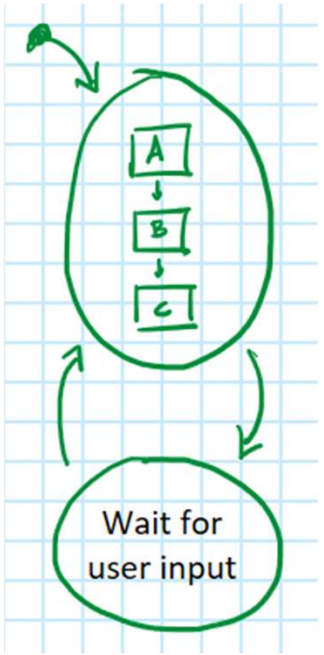

① Abort




② Retry

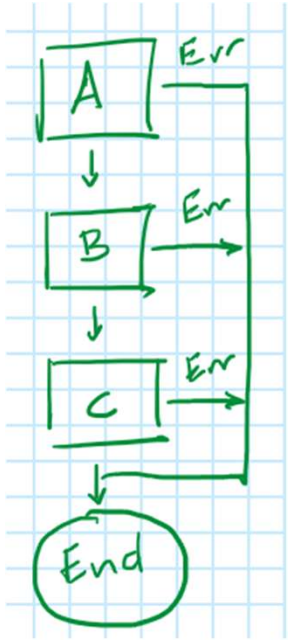



③ Change state



# 6a) Handle the exception: Abort (try/catch)

```
void myTask(void)
{
  while(1)
  {
    try
    {
      uint16_t val = A();
      B(val);
      C();
    }
    catch{...}
  }
}
```



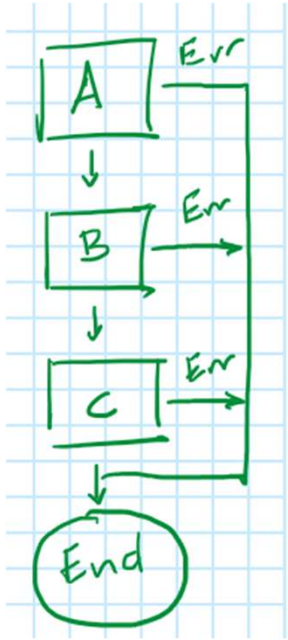
# 6a) Handle the exception: Abort (try/catch)

```
void myTask(void)
{
  while(1)
  {
    try
    {
      uint16_t val = A();
      B(val);
      C();
    }
    catch{...}
  }
}
```

```
uint16_t val;
try{
  try{ val = A(); }
  catch{ throw a_failed; }

  try{ B(val); }
  catch{ throw b_failed; }

  try{ C(); }
  catch{ throw c_failed; }
}
catch{...}
```



# 6a) Handle the exception: Abort (Return values)

## Logical or

```
uint16_t val;
err_t err = NO_ERR;
(err = A(&val)) || (err = B(val)) || (err = C());
```

## Logical and

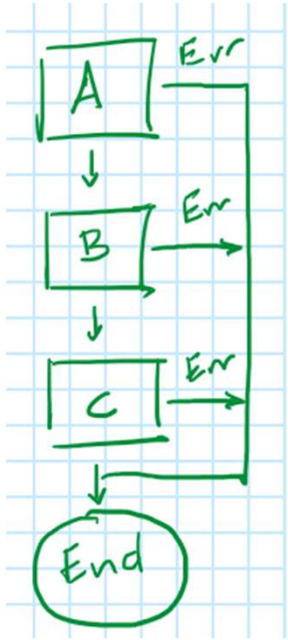
```
!(err = A(&val)) && !(err = B(val)) && !(err = C());
```

## Ternary

```
(err = A(&val)) ? { /* A() failed */ } :
(err = B(val))  ? { /* B() failed */ } :
(err = C())     ? { /* C() failed */ } : { /* No failures */ };
```

## If/else

```
if      (err = A(&val)) { /* A() failed */ }
else if (err = B(val))  { /* B() failed */ }
else if (err = C())     { /* C() failed */ }
else{ /* No failures */ }
```



# 6a) Handle the exception: Abort (Return values)

## Logical or

```
uint16_t val;
err_t err = NO_ERR;
(err = A(&val)) || (err = B(val)) || (err = C());
```

Native short-circuits

## Logical and

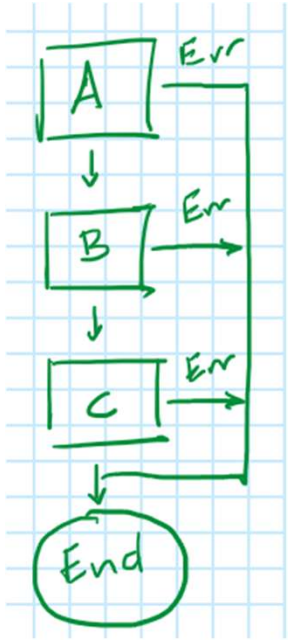
```
!(err = A(&val)) && !(err = B(val)) && !(err = C());
```

## Ternary

```
(err = A(&val)) ? { /* A() failed */ } :
(err = B(val))  ? { /* B() failed */ } :
(err = C())     ? { /* C() failed */ } : { /* No failures */ };
```

## If/else

```
if      (err = A(&val)) { /* A() failed */ }
else if (err = B(val))  { /* B() failed */ }
else if (err = C())     { /* C() failed */ }
else{ /* No failures */ }
```



# 6a) Handle the exception: Abort (Return values)

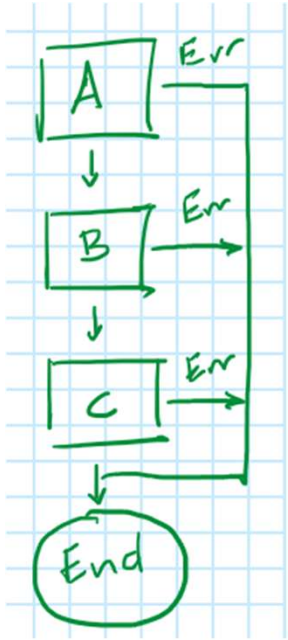
## Do/while

```
uint16_t val;
err_t err = NO_ERR;
do
{
  A(&val); if(err) break;
  B(val);  if(err) break;
  C();
} while(0);
```

General short-circuits

## If guards

```
err = A(&val);
if(!err) err = B(val);
if(!err) err = C();
```



# 6a) Handle the exception: Abort (Return values)

## Guard Clauses

```
err_t foo( ARGS ){
  err_t err = NO_ERR;
  if      (/* error */)          { err = ERR1; }
  else if(/* another error */) { err = ERR2; }
  else if(/* a third error */) { err = ERR3; }
  else
  {
    // Perform foo
  }
  return err;
}
```

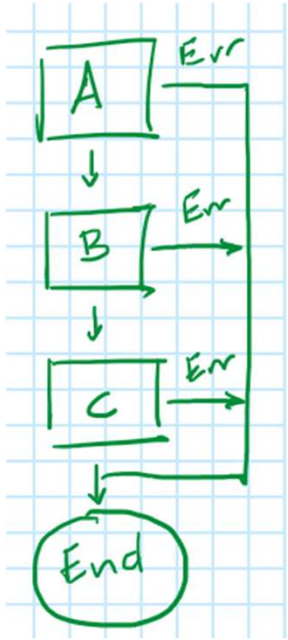
```
err_t foo( ARGS ){
  err_t err = NO_ERR;
  if( !(/* error */)          &&
      !(/* another error */) &&
      !(/* a third error */) )
  {
    // Perform foo
  }
  else err = PRECONDITION_NOT_MET;
  return err;
}
```

# 6a) Handle the exception: Abort (Out-args/Local var)

```
uint16_t val;
err_t err = NO_ERR;
({ expr1; err = A(&val); }) ||
({ expr2; err = B(val); })  ||
({ expr3; err = C(); });

uint16_t val;
err_t err = NO_ERR;
({ val = A(&err); err; }) ||
({ B(&err, val); err; })  ||
C(&err);
```

Statement expression

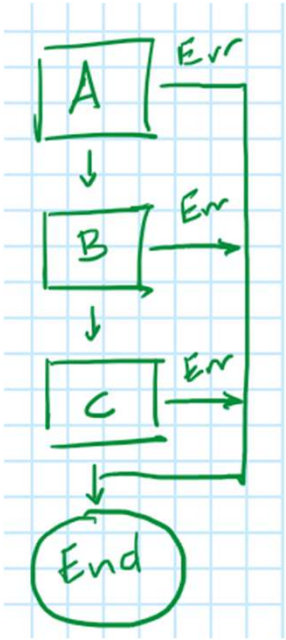


# 6a) Handle the exception: Abort (Out-args/Local var)

```
void myTask(void)
{
  while(1)
  {
    err_t err = NO_ERR;
    uint16_t val = A(&err);
    B(&err, val);
    C(&err);
    if(err){...}
  }
}
```

```
uint16_t A(err_t * err)
{
  if(!(*err))
  {
    // Do the stuff
    if(/* error */) *err = ___;
  }
}
```

Each fcn



# 6a) Handle the exception: Abort (sum types)

std::optional/std::expected & C++23

Has either data or error code

```
std::expected<uint16_t, err_t> A();
std::expected<void, err_t> B(uint16_t val);
std::expected<void, err_t> C();

void myTask(void)
{
  while(1)
  {
    A();
    .and_then([](uint16_t val) { return B(val);      })
    .and_then([]()             { return C();         })
    .or_else  ([](err_t e)     { /* Handle error (/ });
  }
}
```

lambdas

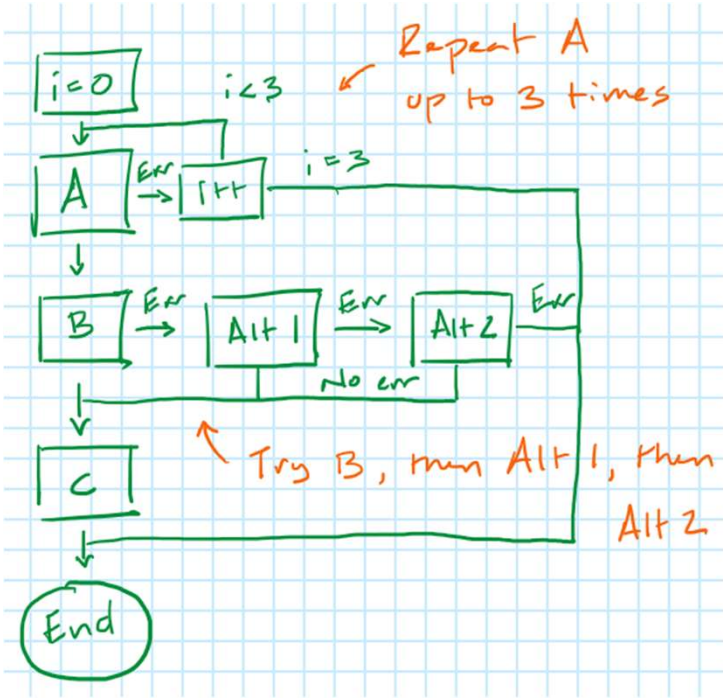
# Your turn!

Implement an “abort”!

- One of your previous/current projects
- From these sample systems
  - https://wokwi.com/projects/430777548539551745
  - https://wokwi.com/projects/430842301912312833
  - https://www.tinkercad.com/things/7TYqSek80uE-eoc2025exception-handling-workshoptinkercad?sharecode=89255xc2210w49tafGJeKD75uj5ZI1LUb-5E1cqrbjE

# 6b) Handle the exception: Retry (try/catch)

```
void myTask(void)
{
  while(1)
  {
    uint16_t val;
    for(size_t count = 3; count > 0; count--)
    {
      try{ val = A(); }
      catch{...}
    }
    try{ B(val); }
    catch{
      try{ alt1(val); }
      catch{
        try{ alt2(val); }
        catch{...}
      }
    }
  }
}
```



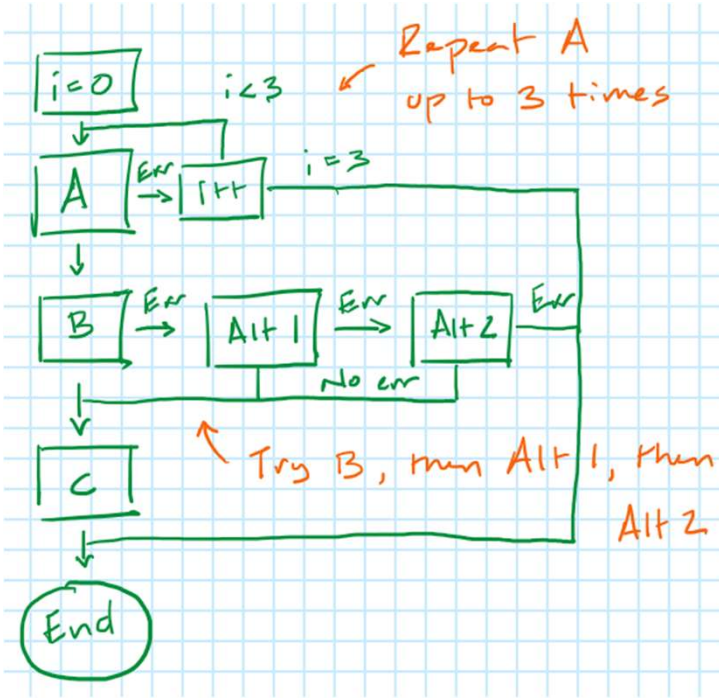
# 6b) Handle the exception: Retry (return values)

```
void myTask(void)
{
  while(1)
  {
    uint16_t val;
    err_t err = NO_ERR;

    ( (err = A(&val)) &&
      (err = A(&val)) &&
      (err = A(&val)) )      ||

    ( (err = B(val)) &&
      (err = altB1(val)) &&
      (err = altB2(val)) )   ||

    for( size_t count = 0;
         (count < 5) && !(err = C());
         count++ ){};
  }
}
```

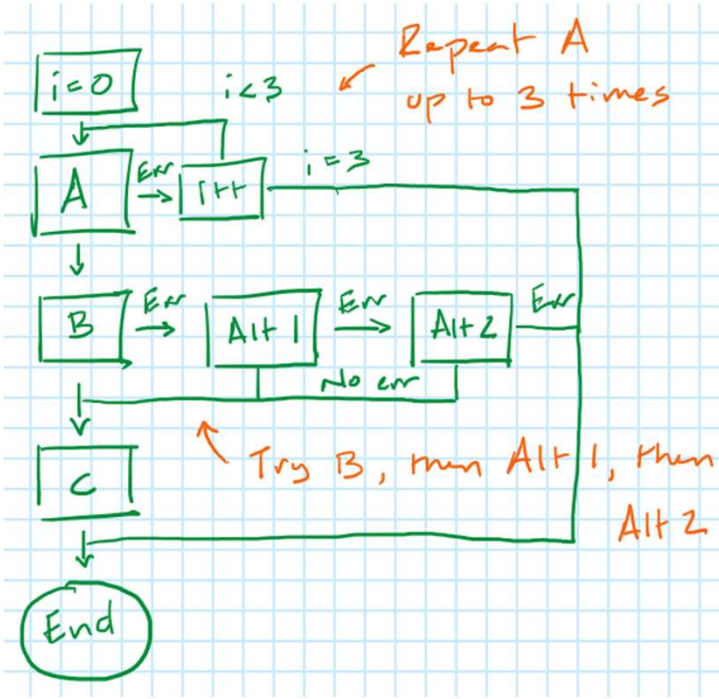


# 6b) Handle the exception: Retry (return values)

```
void myTask(void)
{
  while(1)
  {
    uint16_t val;
    err_t err = NO_ERR;
    (err = tryAThrice(&val))          ||
    (err = tryBThenAlt1ThenAlt2(val)) ||
    (err = tryCFiveTimes());
  }
}
```

Use “Extract Function” liberally!

```
err_t tryAThrice(uint16_t * val)
{
  err_t err = NO_ERR;
  (err = A(val)) && (err = A(val)) && (err = A(val));
  return err;
}
```



# Your turn!

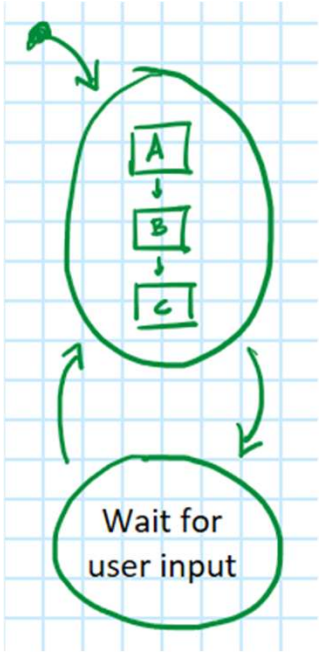
Implement a “retry”!

- One of your previous/current projects
- From these sample systems
  - https://wokwi.com/projects/430777548539551745
  - https://wokwi.com/projects/430842301912312833
  - https://www.tinkercad.com/things/7TYqSek80uE-eoc2025exception-handling-workshoptinkercad?sharecode=89255xc2210w49tafGJeKD75uj5ZI1LUb-5E1cqrbjE

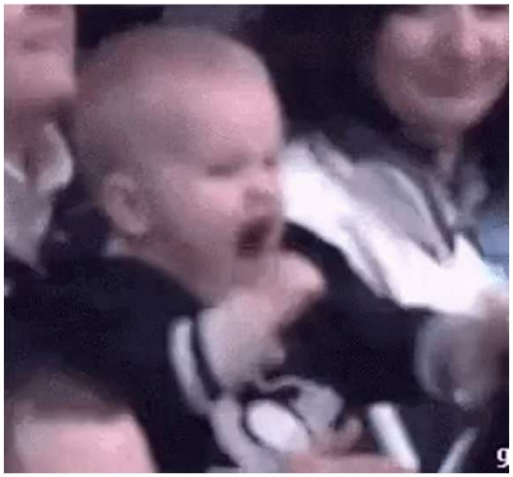
# 6c) Handle the exception: Change state

Ex: After 3 consecutive errors, default to safe state to await user input.

```
void myTask(void){
  uint8_t err_count = 0;
  enum { DEFAULT, SAFE } state = DEFAULT;
  while(1){
    switch(state){
      case DEFAULT:
        uint16_t val;
        err_t err = NO_ERR:
        (err = A(&val)) || (err = B(val)) || (err = C());
        if(err){ err_count++; if(err_count >= 3) state = SAFE; }
        else err_count = 0;
        break;
      case SAFE:
        if(D()){ state = DEFAULT; err_count = 0; }
        break;
    }
  }
}
```

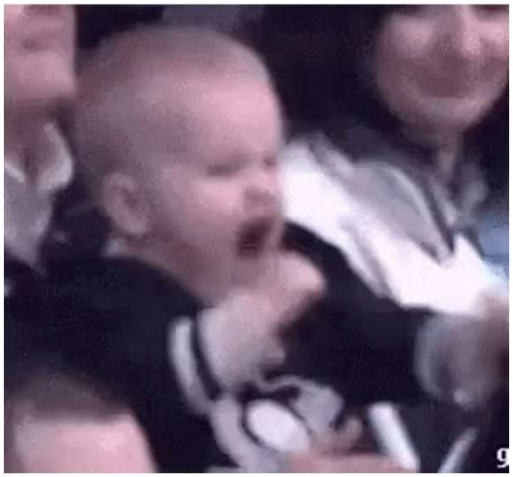


# 7) Celebrate!



# 7) Celebrate!

```
void myTask(void)
{
  /*****************************************
  *  ,_,   Agile Owl says this function is  *
  * (O,O)  actively responding to possible  *
  * (   )  exceptions and won't get derailed *
  *  " "   by the real world!               *
  *****************************************/
  while(1)
  {
    uint16_t val
    err_t err = A(&val);
    err = B(&val);
    err = C();
  }
}
```

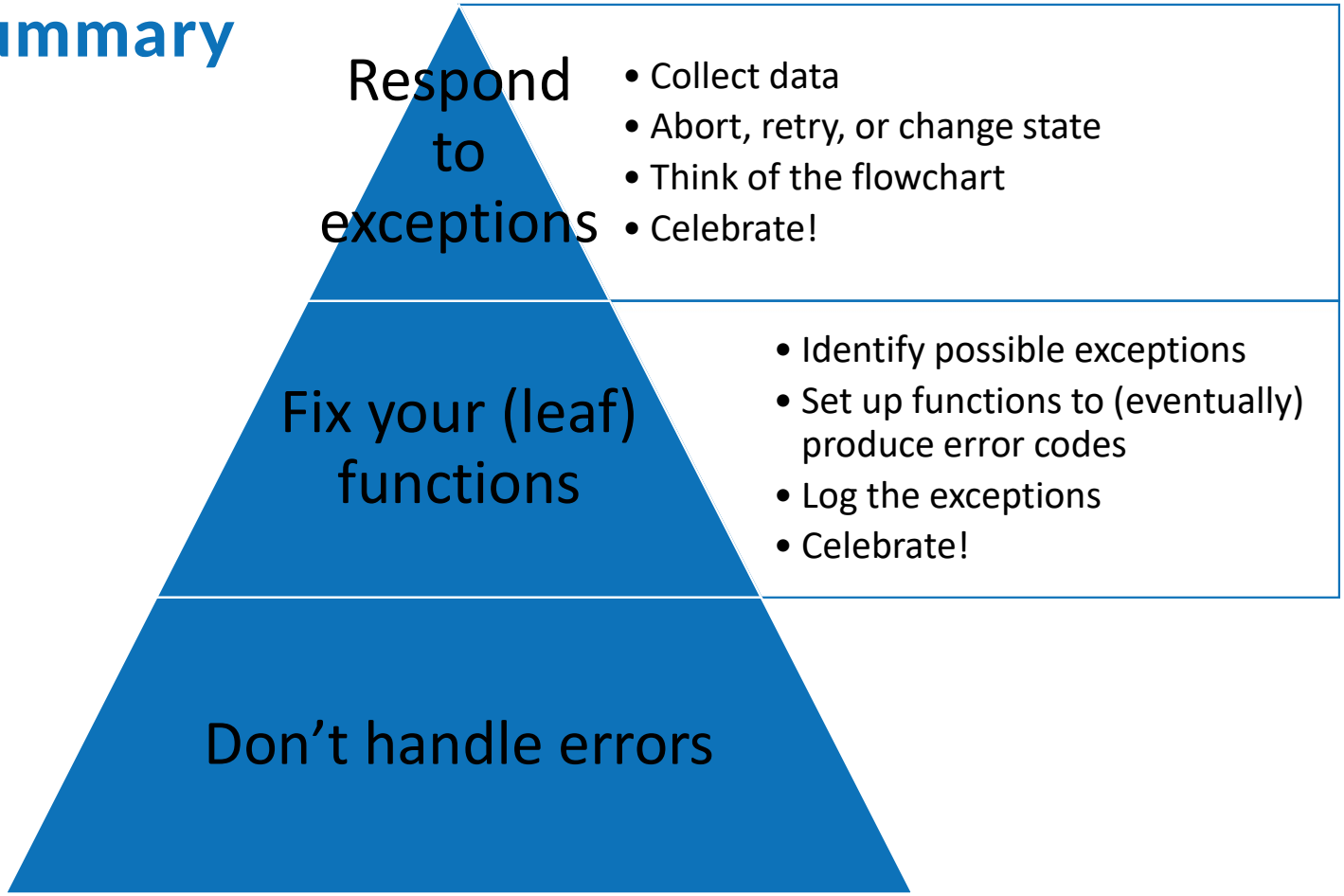


# Your turn!

Change state!

- One of your previous/current projects
- From these sample systems
  - https://wokwi.com/projects/430777548539551745
  - https://wokwi.com/projects/430842301912312833
  - https://www.tinkercad.com/things/7TYqSek80uE-eoc2025exception-handling-workshoptinkercad?sharecode=89255xc2210w49tafGJeKD75uj5ZI1LUb-5E1cqrbjE

# Summary



# THANK YOU

Embedded
Online
Conference

www.embeddedonlineconference.com